Informe de Arquitectura y Avance

Sistema de Análisis de Precio Unitario (APU) para Servicios de Embobinado

El presente documento detalla las decisiones arquitectónicas, tecnológicas y de modelado de datos tomadas durante la fase inicial de desarrollo del Sistema de APU. El objetivo principal es construir una plataforma robusta, escalable y financieramente segura para la cotización de servicios de reparación y mantenimiento de motores y componentes electromecánicos.

1. Justificación del Stack Tecnológico

Para asegurar la estabilidad a largo plazo y facilitar el mantenimiento del sistema en un entorno empresarial, se ha seleccionado un conjunto de tecnologías probadas en la industria:

Java 21 & Spring Boot 3

Elegido por su solidez en aplicaciones backend orientadas a transacciones y cálculos precisos. El tipado estático de Java y la clase `BigDecimal` garantizan que no existan errores de redondeo en cálculos monetarios, un fallo común en lenguajes como JavaScript.

PostgreSQL 16

Motor de base de datos relacional de nivel empresarial. Es ideal para este proyecto debido a su capacidad para manejar relaciones complejas (APUs, listas, catálogos) y realizar cálculos agregados de manera eficiente a nivel de servidor.

Docker (Contenedores)

La base de datos se ejecuta en un contenedor Docker local. Esto aísla el entorno de desarrollo, evitando conflictos con otros servicios de Windows, y permite que el proyecto sea fácilmente desplegado en cualquier otro equipo técnico.

Spring Data JPA (Hibernate)

Actúa como la capa ORM (Object-Relational Mapping). En lugar de escribir sentencias SQL manualmente para operaciones CRUD, JPA gestiona automáticamente la creación de tablas y la hidratación de objetos en memoria, acelerando el desarrollo.

> **Decisión de Diseño: Tipos Numéricos**
>
> Todos los valores financieros y de porcentajes en el sistema utilizan `BigDecimal` con precisiones exactas (ej. 12 dígitos y 2 decimales para costos; 5 decimales para depreciación). Esto evita la pérdida de precisión inherente al uso de datos de punto flotante genéricos (floats/doubles).

2. Arquitectura de Datos y "Snapshots"

El diseño de la base de datos se estructuró para resolver un problema crítico en los sistemas de cotización: **la mutabilidad histórica**. Un APU es un documento legal y financiero. Si los precios base de los materiales o la mano de obra cambian en el futuro, los presupuestos generados meses o años atrás no deben alterarse bajo ninguna circunstancia.

Para resolver esto, el sistema implementa un patrón de *Snapshotting* (captura de estado) dividido en dos capas:

- **Catálogos Maestros (Listas Propias):** Tablas base (`Material`, `Equipment`, `Labor`) donde los usuarios registran descripciones y precios unitarios vigentes una sola vez.
- **Tablas de Procesos (Snapshots):** Tablas transaccionales (`ProcessMaterial`, `ProcessEquipment`, `ProcessLabor`) que vinculan el trabajo específico (`Process`) con el catálogo, pero **copian y congelan** el costo y la descripción en el momento exacto de la creación del APU.

3. Identificadores Manuales

En lugar de delegar la creación de IDs (Claves Primarias) a secuencias automáticas de la base de datos, el modelo de datos requiere que los IDs sean ingresados manualmente por el usuario o el sistema frontend. Esto permite que el software se integre fluidamente con la codificación de inventario o los números de tickets (Solicitudes de Servicio) que el taller ya utiliza en la vida real.

> **Estructura de la Cabecera (Process)**
>
> La tabla principal `Process` centraliza las variables globales del APU. Contiene los metadatos del equipo (potencia, serial, marca), las variables operativas (rendimiento, leyes sociales, porcentaje de administración e imprevistos) y almacena permanentemente la sumatoria final calculada. Esto elimina la necesidad de recalcular todo el presupuesto cada vez que un usuario visualiza el historial.

4. Diagrama Entidad-Relación (ER)

El siguiente diagrama ilustra la estructura relacional actual de la base de datos. Se observa claramente la separación entre la cabecera del proceso, el detalle transaccional (las 3 listas) y los catálogos maestros de referencia.

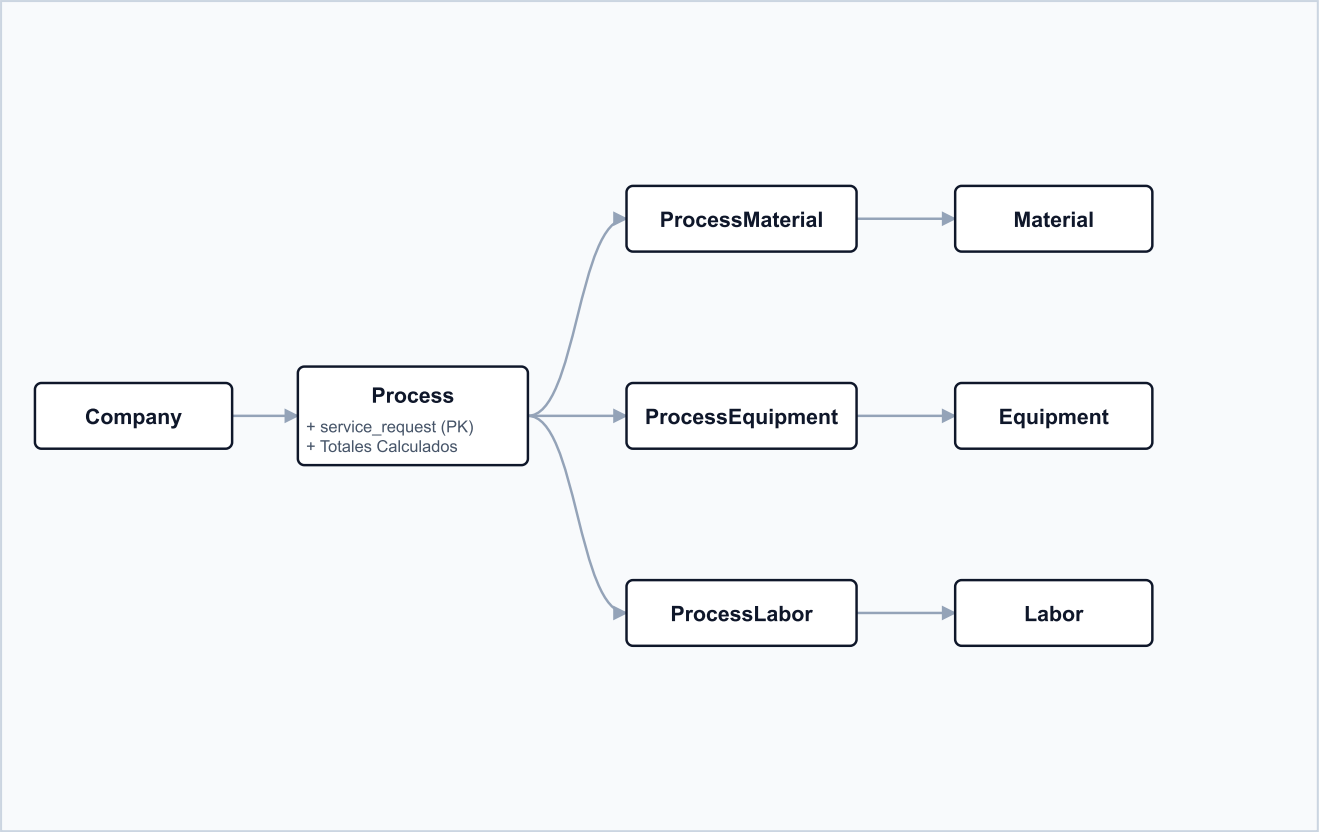


Figura 1. Modelo Relacional Actual (Catálogos, Snapshots y Cabecera)

Próximos Pasos

Con la arquitectura de datos estabilizada y las entidades persistidas en PostgreSQL, el siguiente paso de desarrollo es la creación de la capa de Servicios (`ApuService.java`), donde programaremos el algoritmo matemático encargado de calcular cantidades, depreciaciones, porcentajes y totales antes de generar el presupuesto definitivo.